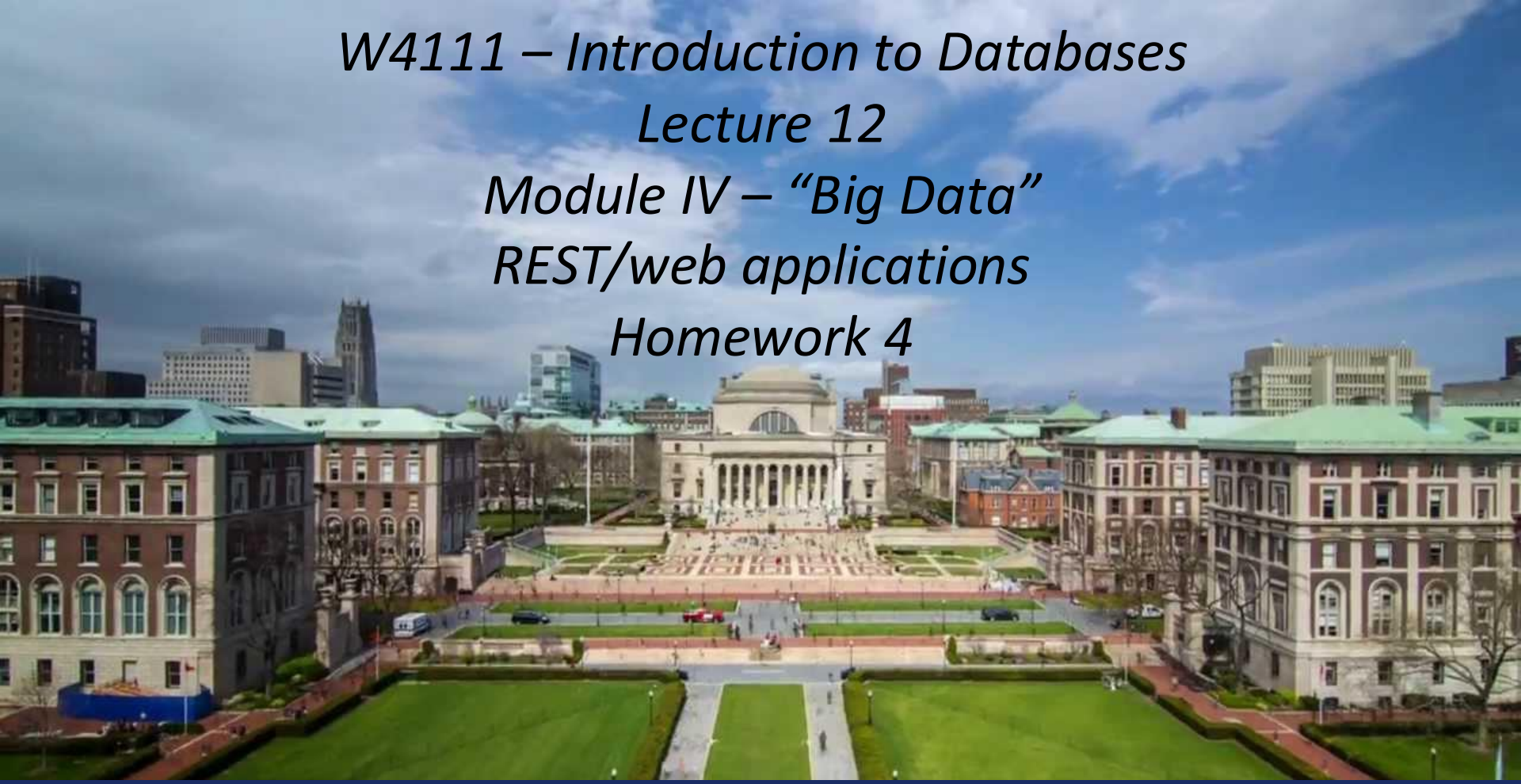
W4111 – Introduction to Databases

Lecture 12

Module IV – "Big Data"

REST/web applications

Homework 4



Contents

Contents

- Course update – schedule for the remainder of the semester.
- Finish module II
  - Concurrent control continued and deadlock
  - Simple demo(s) of transactions and locking is SQL
- Module IV – “Big Data”
  - Core concepts
  - Star schema
  - Data engineering and algebraic operations
- REST/web applications
  - Concepts
  - My application structure/framework
  - Code walkthrough
- Homework 4 – interactive “chalk talk”
  - Non-programming
  - Programming

Course Update

Course Update

- Exam 3
  - Scope is lectures 9, 10, 11 and 12 *plus* normalization. All material in slides and anything I covered in a lectures.
    You *are not* responsible for any book slides.
  - You *will* be responsible for Neo4j query functions explained in the tutorial, and for any MongoDB capabilities covered in lecture, examples, homework 3 and exam 2.
  - The exam will be written questions testing knowledge and understanding of concepts, and some simple queries further testing your understanding of Neo4j and MongoDB.
  - The exam is
    - 01-May from 10:10 to 11:40. Please arrive early to check-in, take seat, etc.
    - The exam is "open book" and "open notes." You may have any written/printed material.
      You may NOT use any electronic devices.
- Homework 4
  - Due 08-May at 11:59 PM.
  - Will *only* be a simple project. There will not be any written questions.
  - There will be a programming track and non-programming track project. You may choose to do either.
  - We will discuss HW4 in this lecture.

Module II
Concurrency Control
Deadlock

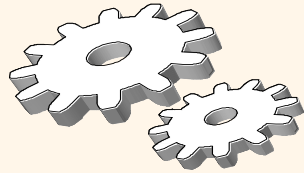
Simple Transaction

```
/*
    Turn of Tx Auto in DataGrip.
    */

-- Set the highest isolation level
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Start the transaction
START TRANSACTION;

-- Example update:
-- Increase total credits by 3 for a specific student
UPDATE student
SET tot_cred = tot_cred + 3
WHERE ID = '00128';

-- Optional: verify the update inside the transaction
SELECT *
FROM student
WHERE ID = '00128';

-- Commit the transaction
COMMIT;
```

To demo:

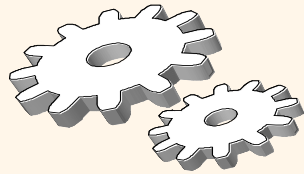
- Locking to enforce serialization
- Examples of COMMIT and ROLLBACK
- Examples of deadlock

Lock-Based Concurrency Control



- *Strict Two-phase Locking (Strict 2PL) Protocol*:
  - Each Xact must obtain a S (*shared*) lock on object before reading, and an X (*exclusive*) lock on object before writing.
  - All locks held by a transaction are released when the transaction completes
    - (Non-strict) 2PL Variant: Release locks anytime, but cannot acquire locks after releasing any lock.
  - If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.
- Strict 2PL allows only serializable schedules.
  - Additionally, it simplifies transaction aborts
  - (Non-strict) 2PL also allows only serializable schedules, but involves more complex abort processing

Aborting a Transaction



- If a transaction *Ti* is aborted, all its actions have to be undone. Not only that, if *Tj* reads an object last written by *Ti*, *Tj* must be aborted as well!
- Most systems avoid such *cascading aborts* by releasing a transaction's locks only at commit time.
  - If *Ti* writes an object, *Tj* can read this only after *Ti* commits.
- In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up.

Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
  1. **exclusive** *(X) mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
  2. **shared** *(S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

Lock-Based Protocols (Cont.)

- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
- But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

Schedule With Lock Grants

- Grants omitted in rest of chapter
  - Assume grant happens just before the next instruction following lock request
- This schedule is not serializable (why?)
- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks.
- Locking protocols enforce serializability by restricting the set of possible schedules.

T_1	T_2	concurrency-control manager
lock-X(B)		
		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		
		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

Deadlock

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

- Neither T_3 nor T_4 can make progress — executing **lock-S**(B) causes T_4 to wait for T_3 to release its lock on B, while executing **lock-X**(A) causes T_3 to wait for T_4 to release its lock on A.
- Such a situation is called a **deadlock**.
  - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

Deadlock (Cont.)

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
  - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
  - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

Deadlock Handling

- System is **deadlocked** if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Deadlock Handling

- ***Deadlock prevention*** protocols ensure that the system will *never* enter into a deadlock state. Some prevention strategies:
  - Require that each transaction locks all its data items before it begins execution (pre-declaration).
  - Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order (graph-based protocol).

More Deadlock Prevention Strategies

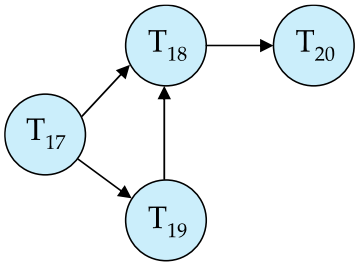
- **wait-die** scheme — non-preemptive
  - Older transaction may wait for younger one to release data item.
  - Younger transactions never wait for older ones; they are rolled back instead.
  - A transaction may die several times before acquiring a lock
- **wound-wait** scheme — preemptive
  - Older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it.
  - Younger transactions may wait for older ones.
  - Fewer rollbacks than *wait-die* scheme.
- In both schemes, a rolled back transactions is restarted with its original timestamp.
  - Ensures that older transactions have precedence over newer ones, and starvation is thus avoided.

Deadlock prevention (Cont.)

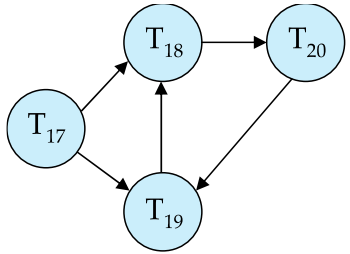
- **Timeout-Based Schemes**:
  - A transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.
  - Ensures that deadlocks get resolved by timeout if they occur
  - Simple to implement
  - But may roll back transaction unnecessarily in absence of deadlock
    - Difficult to determine good value of the timeout interval.
  - Starvation is also possible

Deadlock Detection

- **Wait-for graph**
  - *Vertices:* transactions
  - *Edge from* $T_i \rightarrow T_j$. : if T_i is waiting for a lock held in conflicting mode by T_j
- The system is in a deadlock state if and only if the wait-for graph has a cycle.
- Invoke a deadlock-detection algorithm periodically to look for cycles.



Wait-for graph without a cycle



Wait-for graph with a cycle

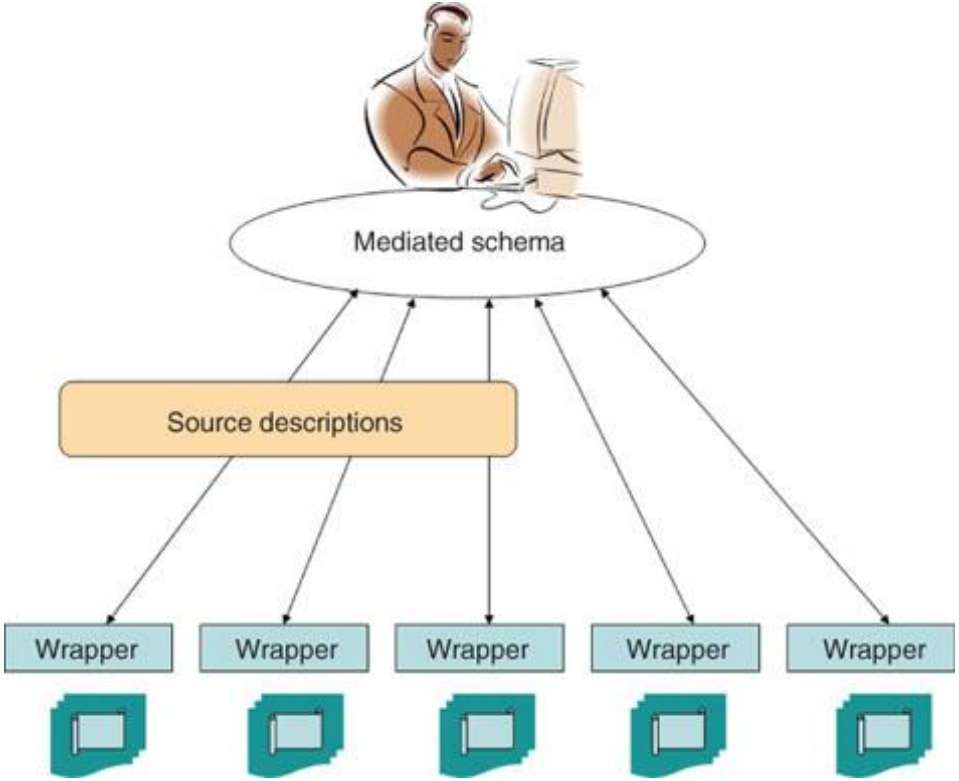
Deadlock Recovery

- When deadlock is detected :
  - Some transaction will have to rolled back (made a **victim**) to break deadlock cycle.
    - Select that transaction as victim that will incur minimum cost
  - Rollback -- determine how far to roll back transaction
    - **Total rollback**: Abort the transaction and then restart it.
    - **Partial rollback**: Roll back victim transaction only as far as necessary to release locks that another transaction in cycle is waiting for
- Starvation can happen (why?)
  - One solution: oldest transaction in the deadlock set is never chosen as victim

Big Data

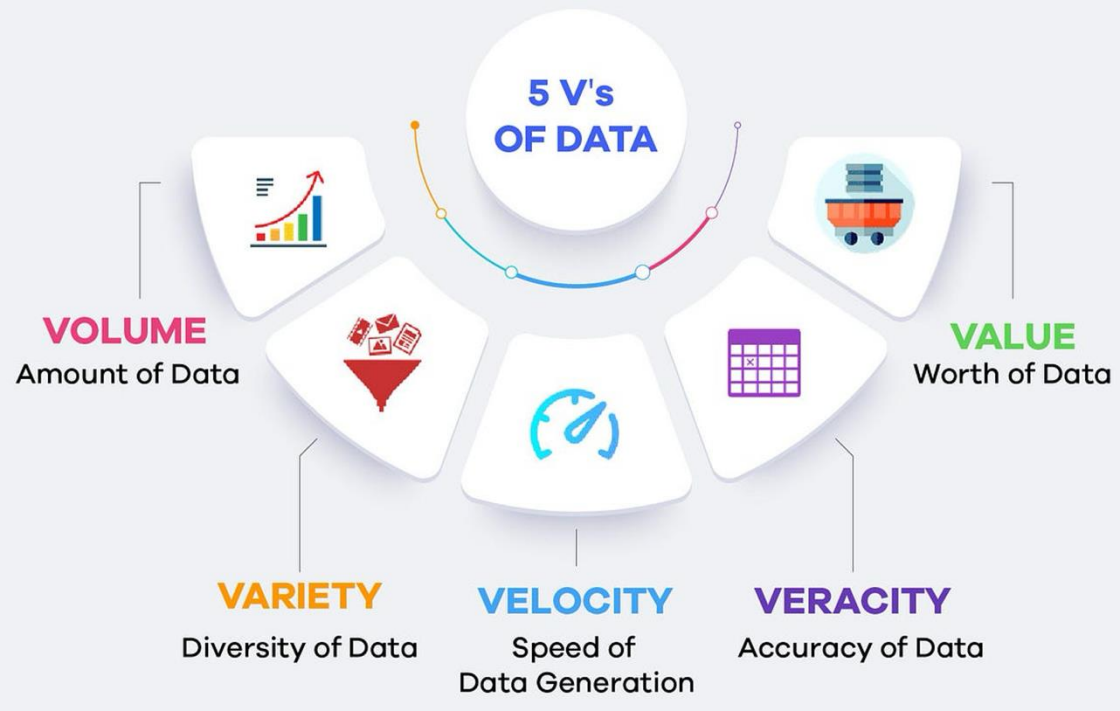
Core Concepts

Enterprise Information Integration



5 V's of Big Data

https://www.linkedin.com/pulse/big-data-management-revolutio-parichehr-esmailian/



https://medium.com/@get_excelsior/big-data-explained-the-5v-s-of-data-ae80cbe8ded1

Types Of Big Data

Following are the types of Big Data:

1. **Structured**
2. **Unstructured**
3. **Semi-structured**

Structured

Any data that can be stored, accessed and processed in the form of fixed format is termed as a 'structured' data. Over the period of time, talent in computer science has achieved greater success in developing techniques for working with such kind of data (where the format is well known in advance) and also deriving value out of it. However, nowadays, we are foreseeing issues when a size of such data grows to a huge extent, typical sizes are being in the rage of multiple zettabytes.

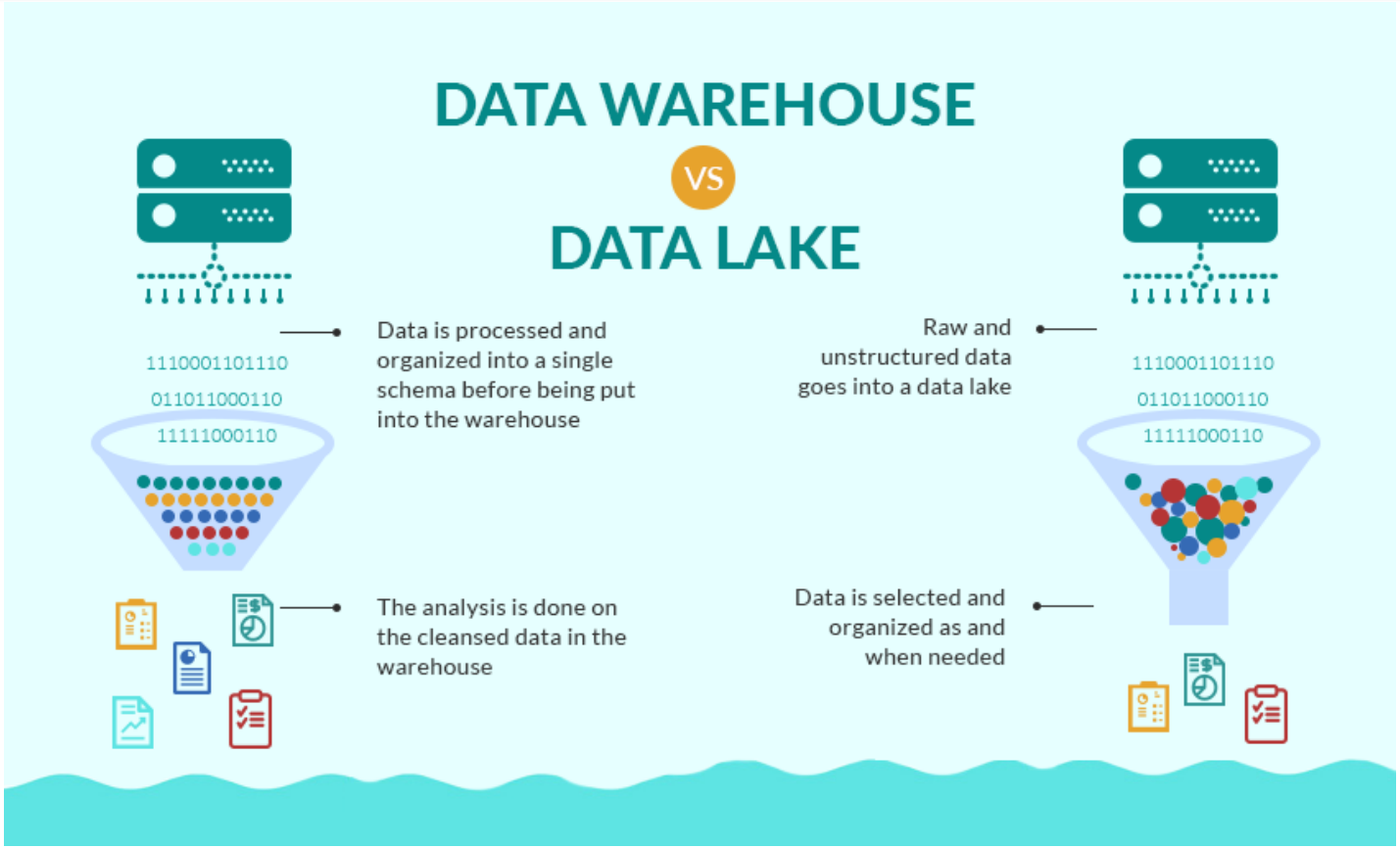
Unstructured

Any data with unknown form or the structure is classified as unstructured data. In addition to the size being huge, un-structured data poses multiple challenges in terms of its processing for deriving value out of it. A typical example of unstructured data is a heterogeneous data source containing a combination of simple text files, images, videos etc. Now day organizations have wealth of data available with them but unfortunately, they don't know how to derive value out of it since this data is in its raw form or unstructured format.

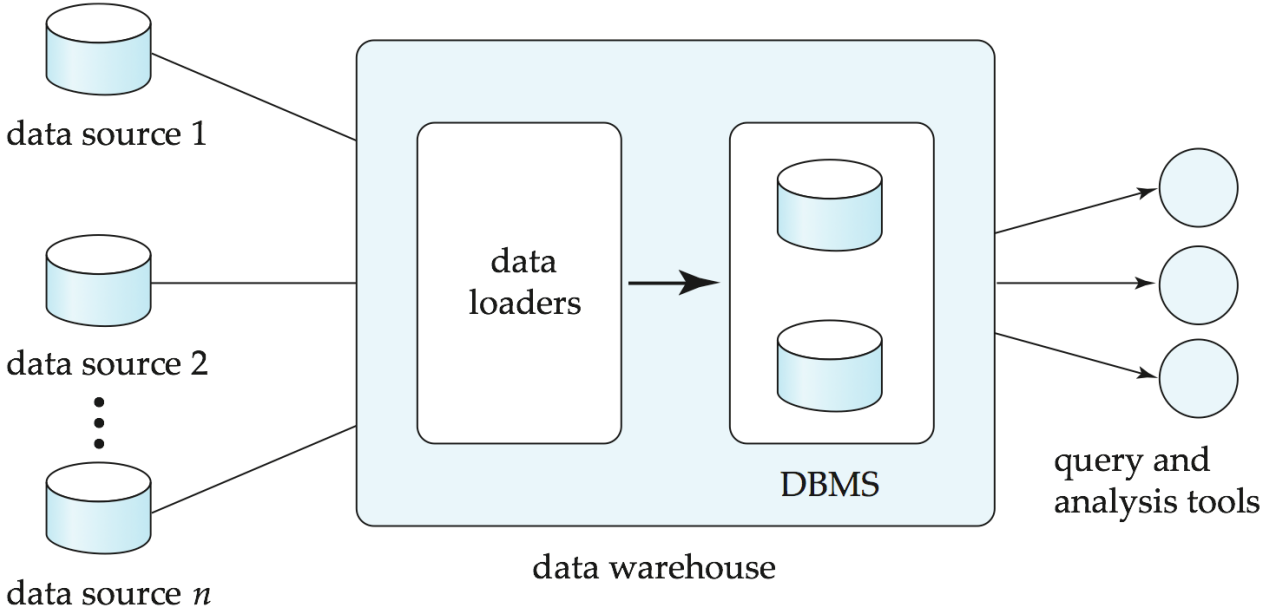
Semi-structured

Semi-structured data can contain both the forms of data. We can see semi-structured data as a structured in form but it is actually not defined with e.g. a table definition in relational DBMS. Example of semi-structured data is a data represented in an XML file.

Data Warehouse and Data Lake



Data Warehousing

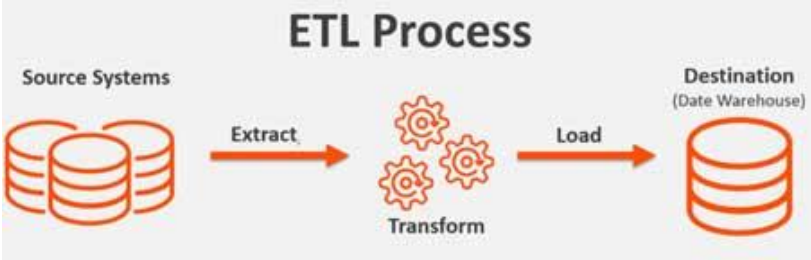


Overview (Cont.)

- Common steps in data analytics
  - Gather data from multiple sources into one location
    - Data warehouses also integrated data into common schema
    - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
      - Can be done as **ETL (extract-transform-load)**, or **ELT (extract-load-transform)**
  - Generate aggregates and reports summarizing data
    - Dashboards showing graphical charts/reports
    - **Online analytical processing (OLAP) systems** allow interactive querying
    - Statistical analysis using tools such as R/SAS/SPSS
      - Including extensions for parallel processing of big data
  - Build **predictive models** and use the models for decision making

ETL Concepts

https://databricks.com/glossary/extract-transform-load



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three Data Extraction methods:**

1. Partial Extraction – The easiest way to obtain the data is if the if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification - Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

- Basic transformations:
- Cleaning
- Format revision
- Data threshold validation checks
- Restructuring
- Deduplication
- Advanced transformations:
- Filtering
- Merging
- Splitting
- Derivation
- Summarization
- Integration
- Aggregation
- Complex data validation

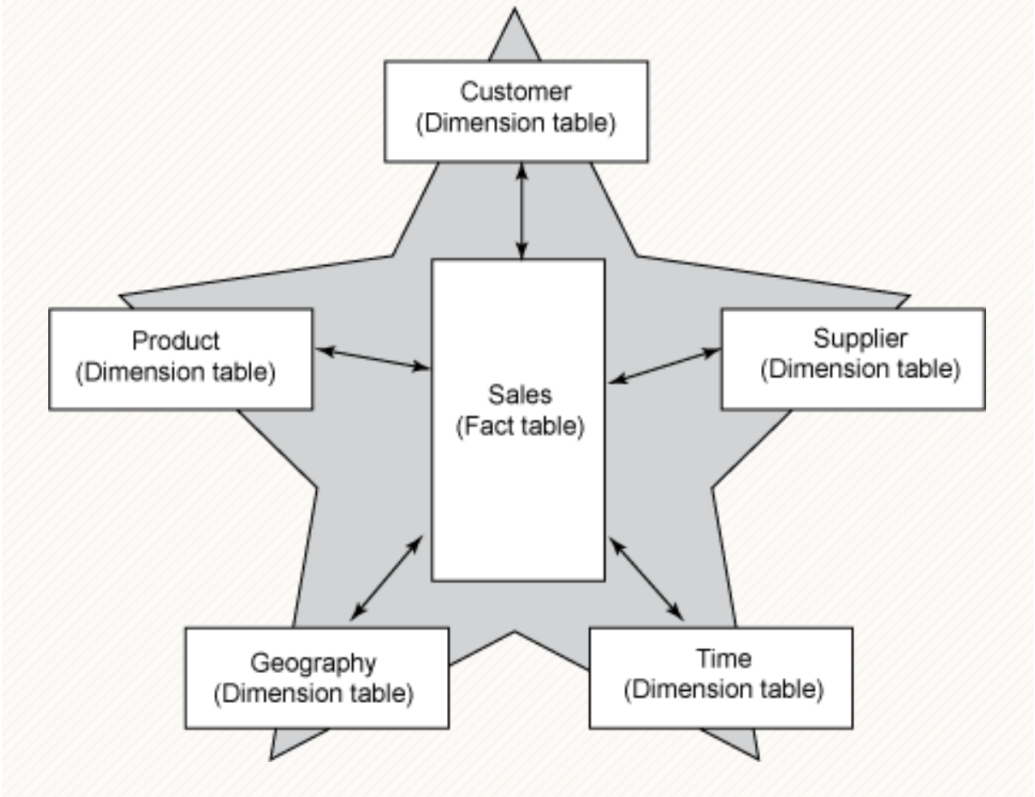
Load

Finally, the load function s is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

Star Schema

Facts and Dimensions

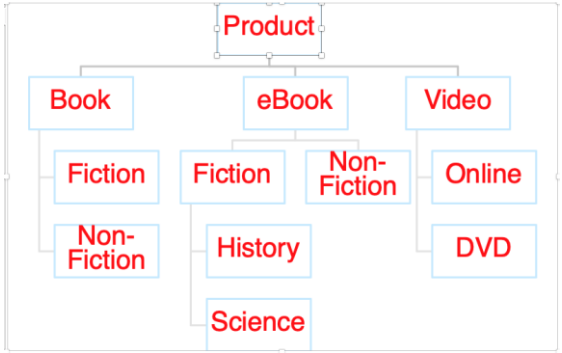
- A common model for (some) data in a date warehouse is a *star schema.*
- There are one or more *fact tables* that have records for small facts, e.g.
  - Sold product A.
  - To customer B.
  - At price C.
- Facts are positioned in dimensions, e.g.
  - Date, time
  - Location
  - Product category
  - … …
- Queries ask for summations and aggregations in dimensions, e.g.
  - Total sales of products in category X, to customers in country Y during year Z.
  - Queries can roll-up, drill down, pivot, … …
- This is a generalization of the spreadsheet concept of *pivot tables.*

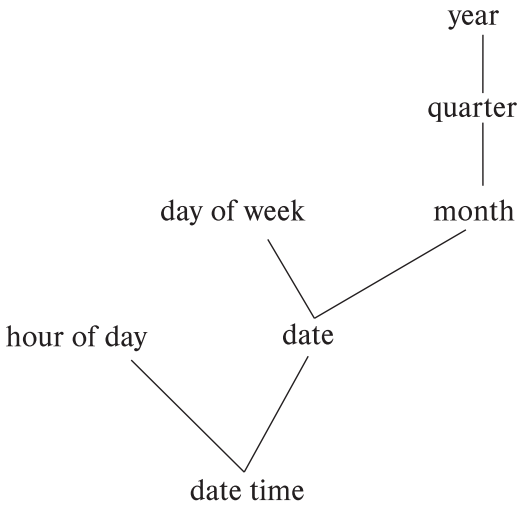


Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
- E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year

Another dimension could be ...
Product category.





(a) time hierarchy

(b) location hierarchy

OLAP

Data Analysis and OLAP

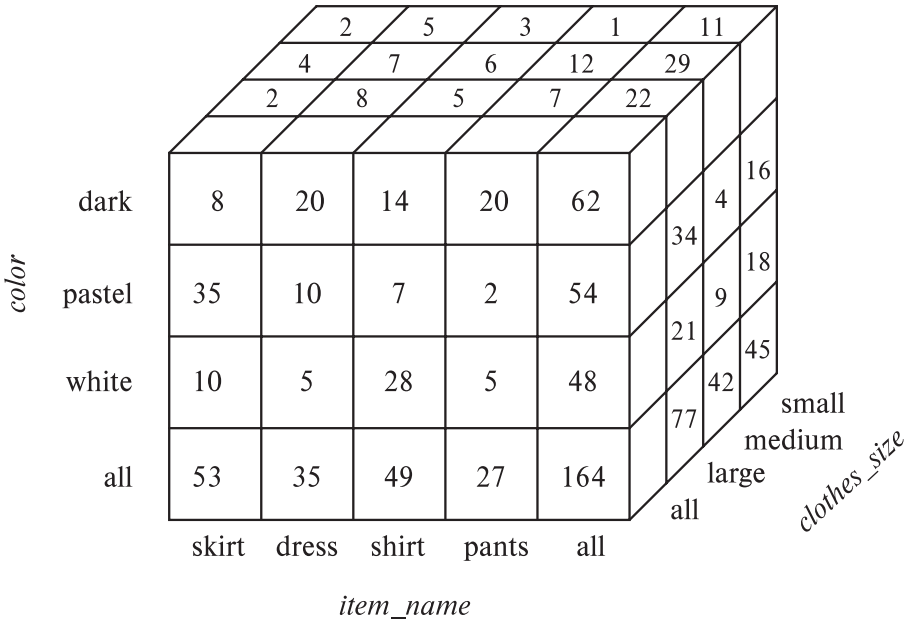
- **Online Analytical Processing (OLAP)**
  - Interactive analysis of data, allowing data to be summarized and viewed in different ways in an online fashion (with negligible delay)
- We use the following relation to illustrate OLAP concepts
  - *sales* (*item_name*, *color*, *clothes_size*, *quantity*)

  This is a simplified version of the *sales* fact table joined with the dimension tables, and many attributes removed (and some renamed)

- For Classic Models, this would be:
  - sales(productCode, quantityOrdered, priceEach)
  - But, I need to link with dimensions.
    - Date
    - Location
    - ProductType
  - ➔
  - sales(productCode, quantityOrdered, priceEach,
    date_dimension_id, location_dimension_id, product_type_dimension_id)
  - We are only going to worry about date and location for now.

Data Cube

- A **data cube** is a multidimensional generalization of a cross-tab
- Can have n dimensions; we show 3 below
- Cross-tabs can be used as views on a data cube



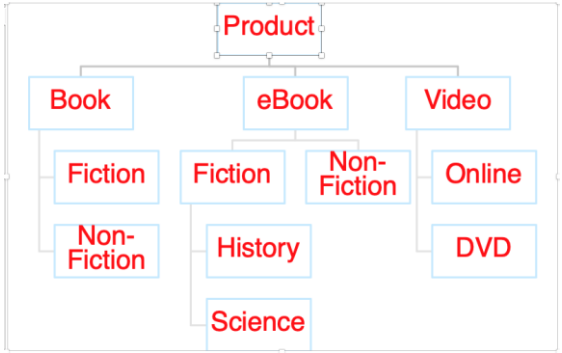
Online Analytical Processing Operations

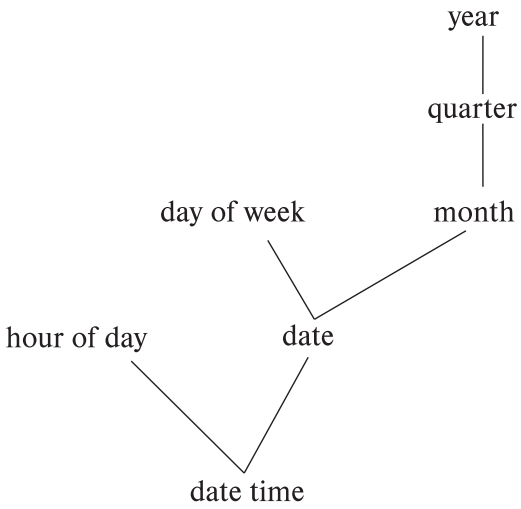
- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
  - E.g., fixing color to white and size to small
  - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.
- **Rollup:** moving from finer-granularity data to a coarser granularity
  - E.g., aggregating away an attribute
  - E.g., moving from aggregates by day to aggregates by month or year
- **Drill down:** The opposite operation - that of moving from coarser-granularity data to finer-granularity data

Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
- E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year

Another dimension could be ...
Product category.





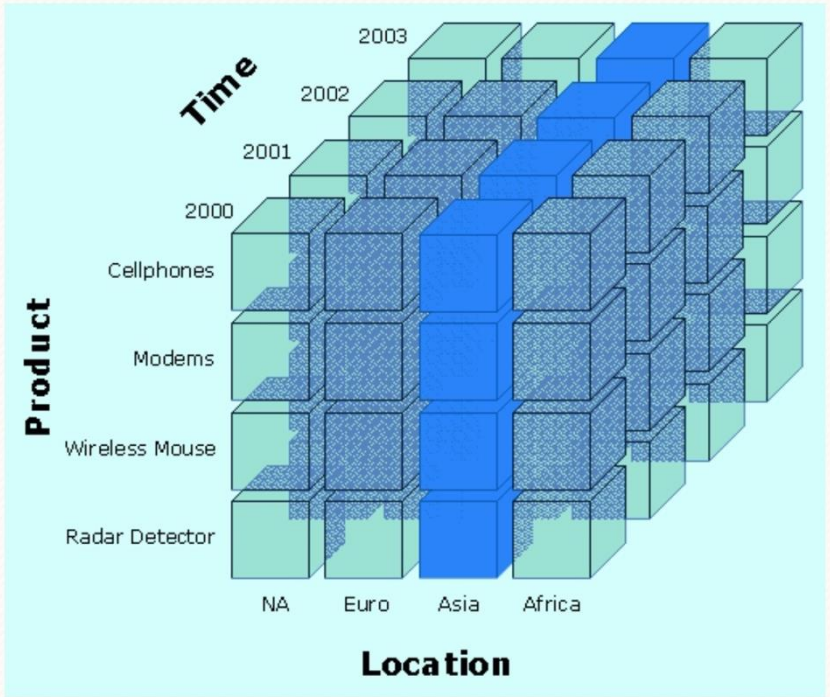
(a) time hierarchy

(b) location hierarchy

Slice

Slice:

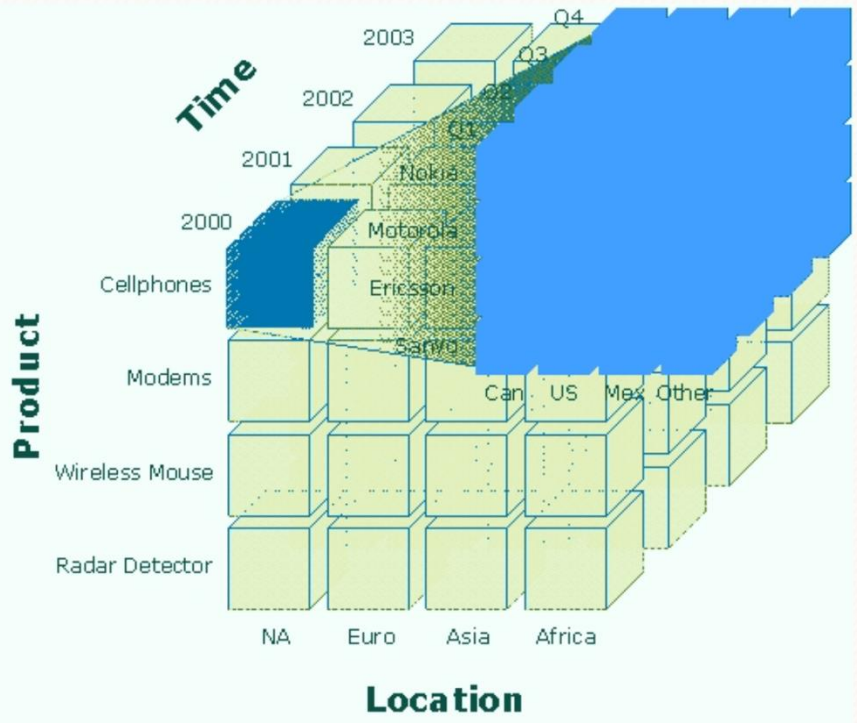
A slice is a subset of a multi-dimensional array corresponding to a single value for one or more members of the dimensions not in the subset.



Dice

Dice:

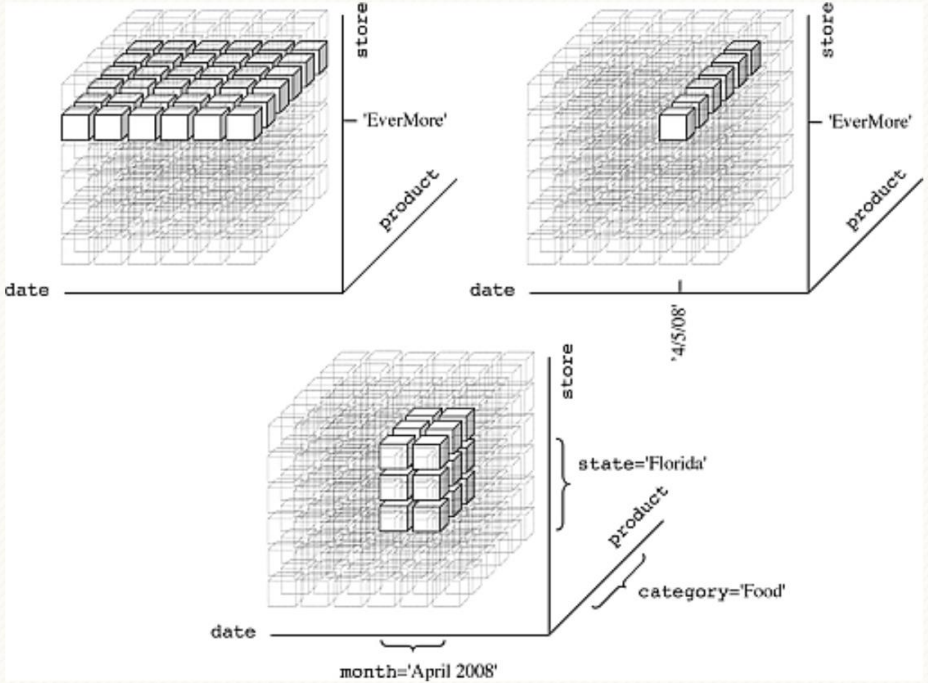
The dice operation is a slice on more than two dimensions of a data cube (or more than two consecutive slices).



Drilling

Drill Down/Up:

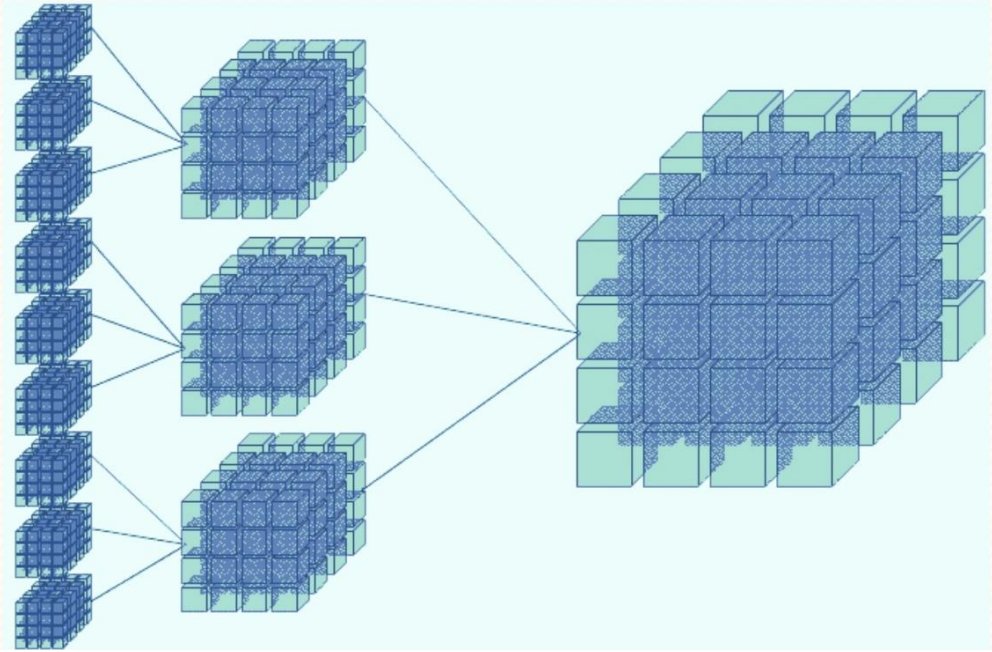
Drilling down or up is a specific analytical technique whereby the user navigates among levels of data ranging from the most summarized (up) to the most detailed (down).



Roll-up

Roll-up:

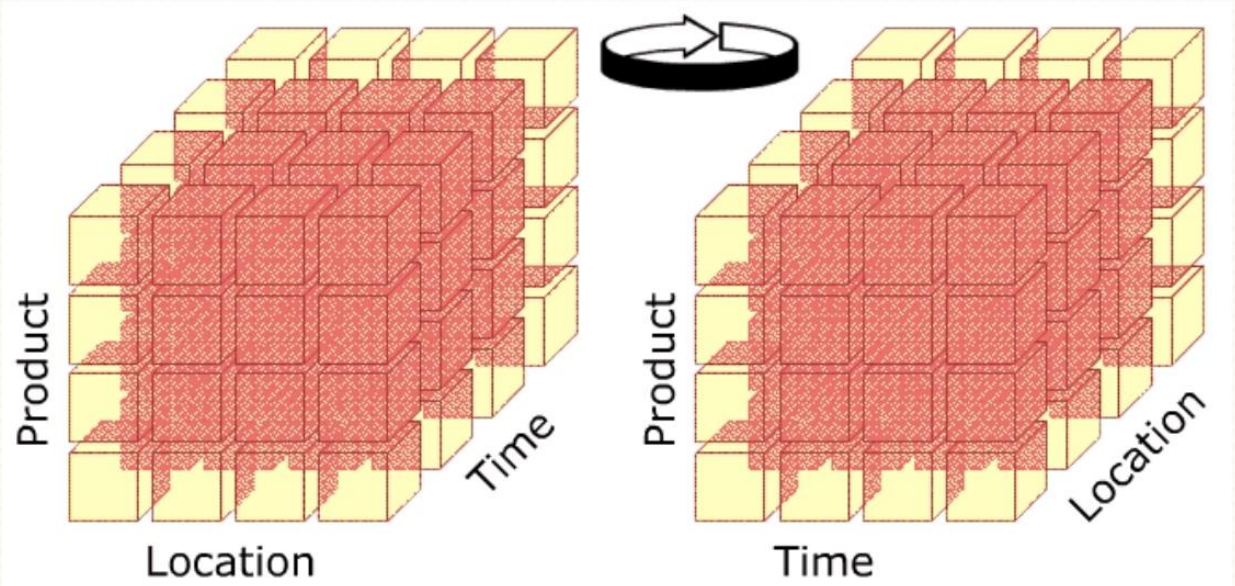
(Aggregate, Consolidate) A roll-up involves computing all of the data relationships for one or more dimensions. To do this, a computational relationship or formula might be defined.



Pivot

Pivot:

This operation is also called rotate operation. It rotates the data in order to provide an alternative presentation of data – the report or page display takes a different dimensional orientation.

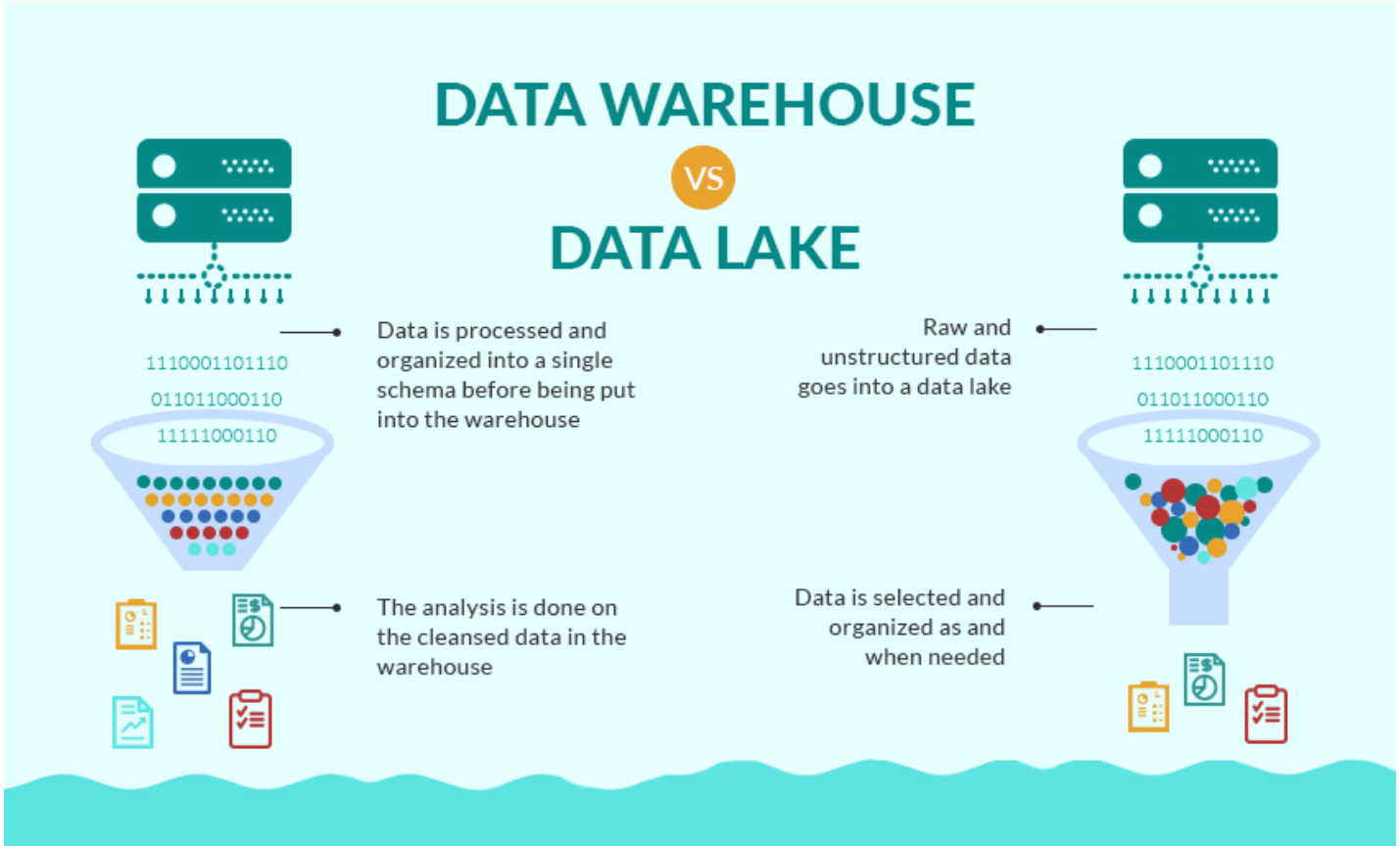


Summary

- OLAP is a very useful model for analyzing business data, typically data that is inherently tabular to begin with.
- There are pure OLAP databases, but a more common approach is:
  - Use a relational database.
  - Define a star schema.
  - ETL from the operational data into the star schema.
  - “Translate” the logical OLAP operations into SQL queries and predefined views.

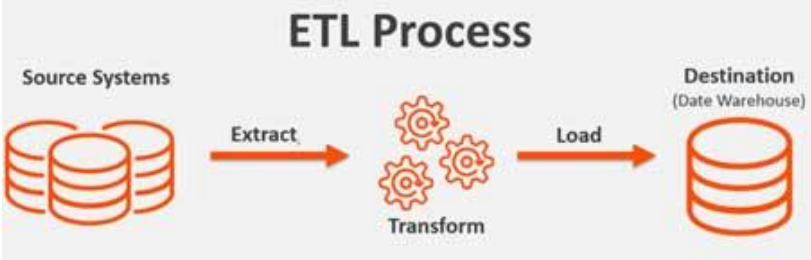
Data Engineering

Data Warehouse and Data Lake



ETL Concepts

https://databricks.com/glossary/extract-transform-load



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three Data Extraction methods:**

1. Partial Extraction – The easiest way to obtain the data is if the if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

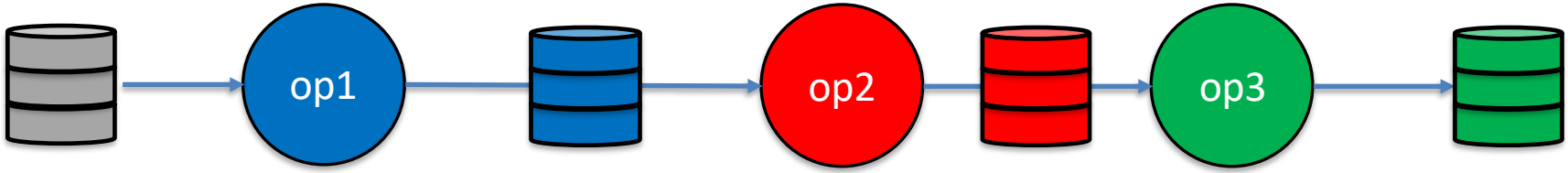
Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

- Basic transformations:
- Cleaning
- Format revision
- Data threshold validation checks
- Restructuring
- Deduplication
- Advanced transformations:
- Filtering
- Merging
- Splitting
- Derivation
- Summarization
- Integration
- Aggregation
- Complex data validation

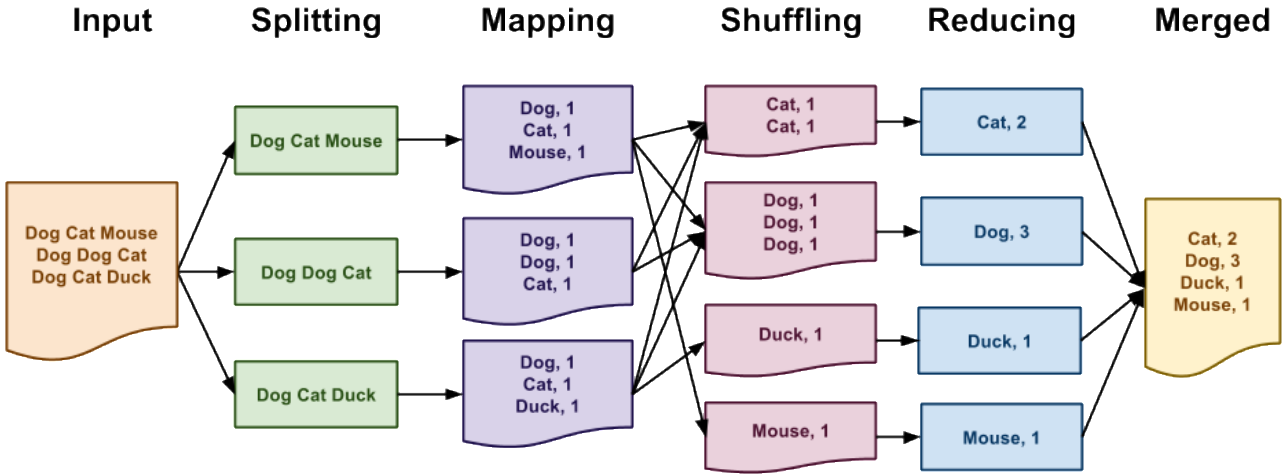
Load

Finally, the load function s is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

MapReduce is a data flow program with relatively simple operators on the data set.



With each operator implemented in parallel on multiple nodes for performance.

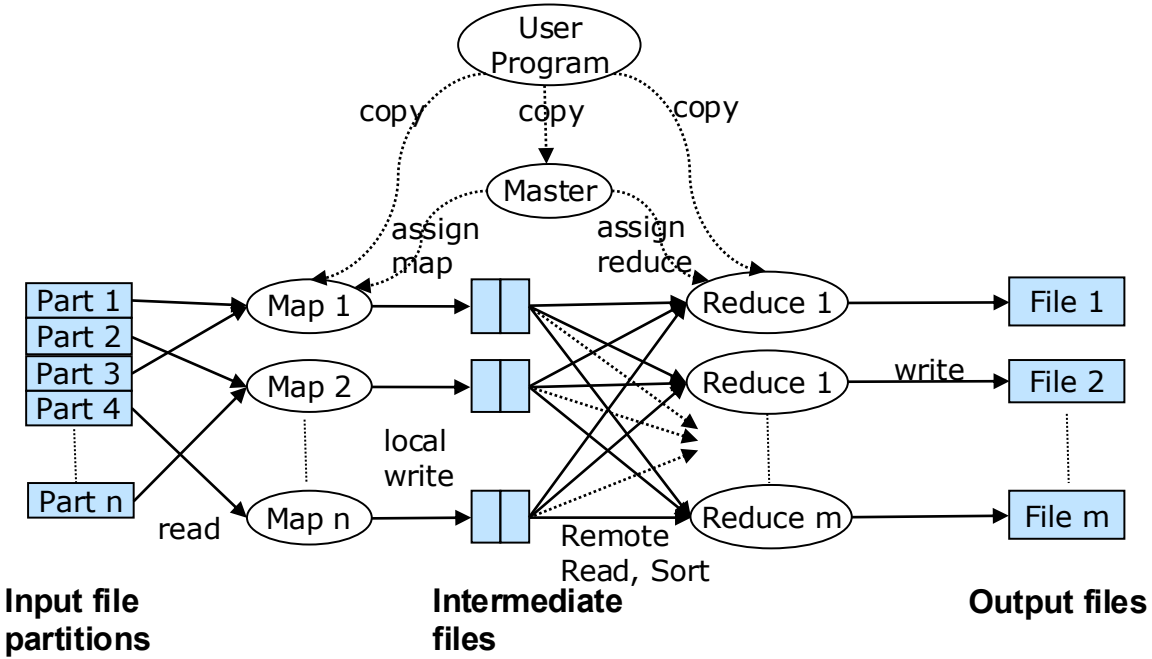


What we want more complex “operators?”

Map Reduce vs. Databases

- Map Reduce widely used for parallel processing
  - Google, Yahoo, and 100's of other companies
  - Example uses: compute PageRank, build keyword indices, do data analysis of web click logs, ….
  - Allows procedural code in map and reduce functions
  - Allows data of any type
- Many real-world uses of MapReduce cannot be expressed in SQL
- But many computations are much easier to express in SQL
  - Map Reduce is cumbersome for writing simple queries

Parallel Processing of MapReduce Job



Map Reduce vs. Databases (Cont.)

- Relational operations (select, project, join, aggregation, etc.) can be expressed using Map Reduce
- SQL queries can be translated into Map Reduce infrastructure for execution
  - Apache Hive SQL, Apache Pig Latin, Microsoft SCOPE
- Current generation execution engines support not only Map Reduce, but also other algebraic operations such as joins, aggregation, etc. natively.

The Value of Blocks

- The “normal” file system abstraction is a stream.
- Consider a very large CSV file:
  - We can read using the normal file system functions and APIs,
  - But we have no idea where records start/end, and
  - Which records on on which blocks.
  - ➔
  - Parallel processing of data in a file is very hard using the stream model.
- Databases, big data tools, etc. surface concepts of record size, block I/Os, mapping of and organization of records to blocks, block location, … … ➔
  - Can exploit module processors to parallel process large datasets.
  - Organize the data to optimize processing, file I/Os, etc.

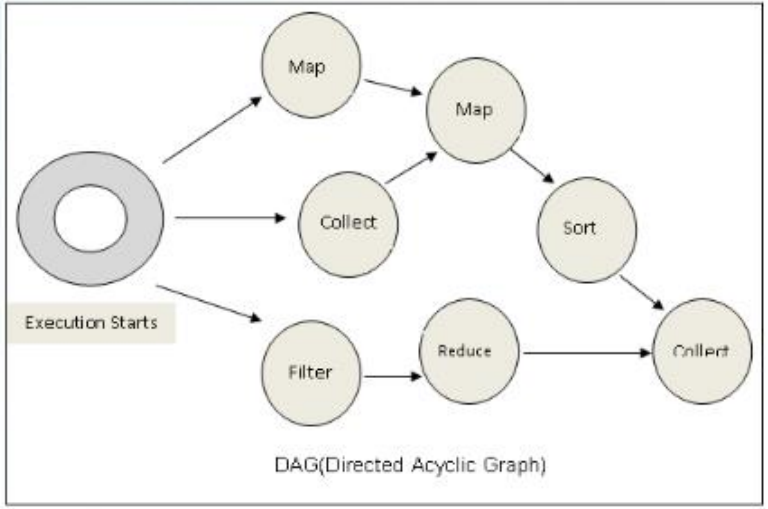
Algebraic Operations

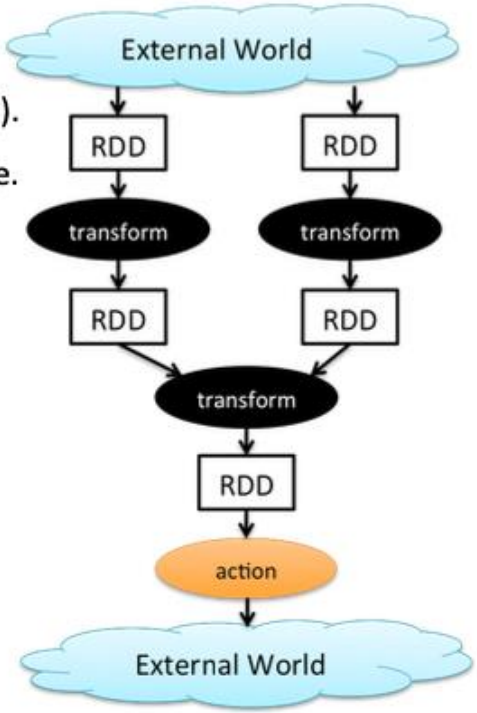
Algebraic Operations

- Current generation execution engines
  - Natively support algebraic operations such as joins, aggregation, etc. natively.
  - Allow users to create their own algebraic operators
  - Support trees of algebraic operators that can be executed on multiple nodes in parallel
- E.g. Apache Tez, Spark
  - Tex provides low level API; Hive on Tez compiles SQL to Tez
  - Spark provides more user-friendly API

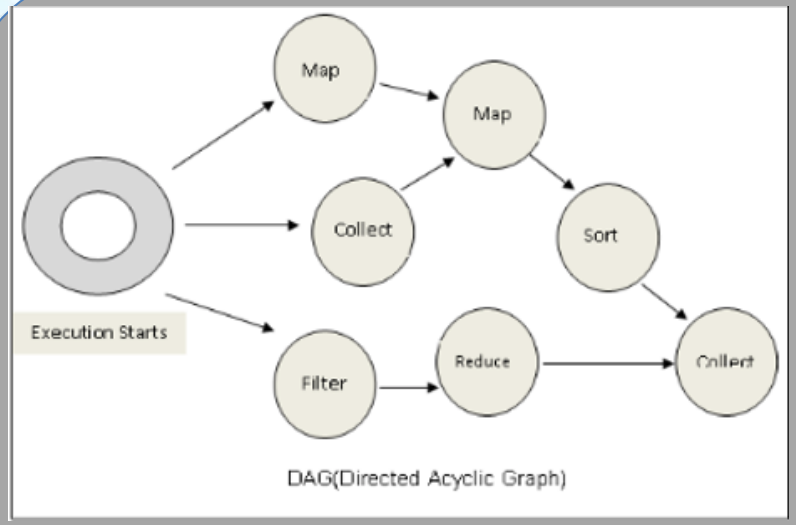
- In the relational model,
  - The are relations.
  - A fixed set of operators that produce relations from relations (closed).
  - Are declarative languages and compute the execution graph/plan.
- The Spark/... ... engines:
  - RDDs, Data Frames
  - Enable programmers to develop and add new operators.
  - Operators convert reliable distributed datasets/data frames to new RDDs/data frames.
  - Data engineer explicitly defines the execution graph (data flow).

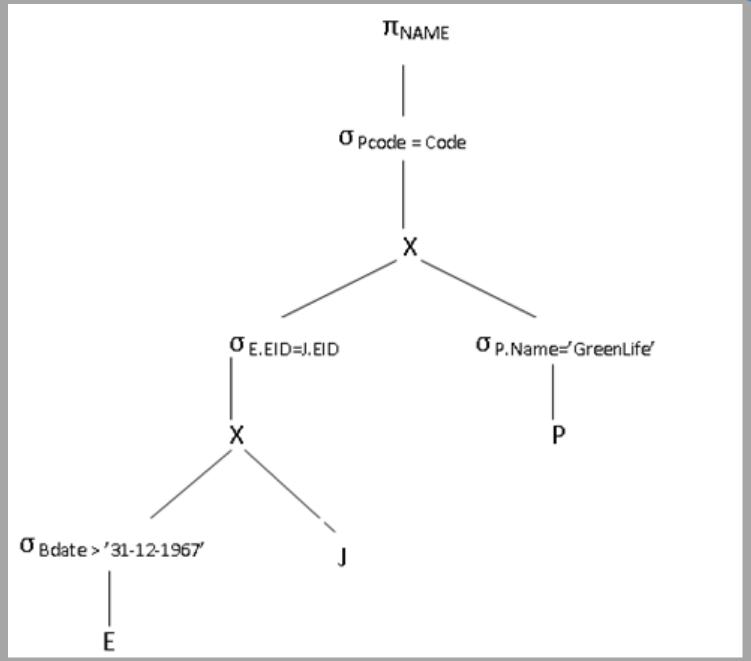
- All jobs in spark comprise a series of operators and run on a set of data.
- All the operators in a job are used to construct a DAG (Directed Acyclic Graph).
- The DAG is optimized by rearranging and combining operators where possible.





www.edureka.co/apache-spark-scala-training





Conceptually similar to query evaluation graph, but ...

- You explicitly define the graph.
- You can develop your own operators.

www.edureka.co/apache-spark-scala-training

Algebraic Operations in Spark

- **Resilient Distributed Dataset** (**RDD**) abstraction
  - Collection of records that can be stored across multiple machines
- RDDs can be created by applying algebraic operations on other RDDs
- RDDs can be lazily computed when needed
- Spark programs can be written in Java/Scala/R
  - Our examples are in Java
- Spark makes use of Java 8 Lambda expressions; the code

  s - > Arrays.asList(s.split(" ")).iterator()

  defines unnamed function that takes argument s and executes the expression Arrays.asList(s.split(" ")).iterator() on the argument
- Lambda functions are particularly convenient as arguments to map, reduce and other functions

Spark/PySpark

DataFrame

edureka!

- Inspired by DataFrames in R and Python (Pandas).
- DataFrames API is designed to make big data processing on tabular data easier.
- DataFrame is a distributed collection of data organized into named columns.
- Provides operations to filter, group, or compute aggregates, and can be used with Spark SQL.
- Can be constructed from structured data files, existing RDDs, tables in Hive, or external databases.

DataFrame features

edureka!

- Ability to scale from KBs to PBs
- Support for a wide array of data formats and storage systems
- State-of-the-art optimization and code generation through the spark SQL catalyst optimizer
- Seamless integration with all big data tooling and infrastructure via spark
- APIs for Python, Java, Scala, and R

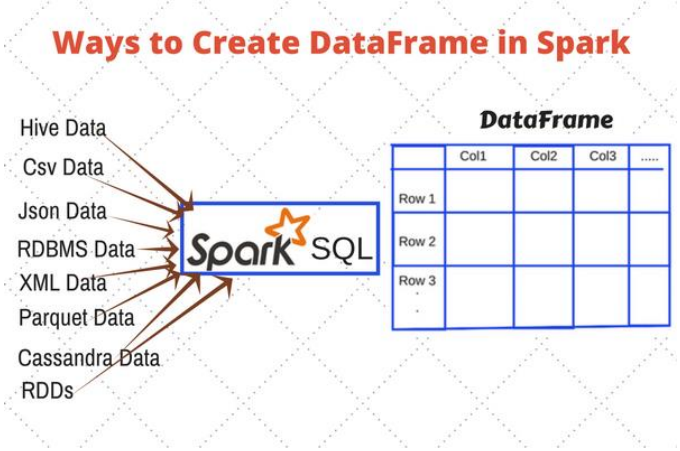
1. DataFrame in PySpark: Overview

In Apache Spark, a DataFrame is a distributed collection of rows under named columns. In simple terms, it is same as a table in relational database or an Excel sheet with Column headers. It also shares some common characteristics with RDD:

- **Immutable in nature** : We can create DataFrame / RDD once but can't change it. And we can transform a DataFrame / RDD after applying transformations.
- **Lazy Evaluations:** Which means that a task is not executed until an action is performed.
- **Distributed:** RDD and DataFrame both are distributed in nature.

My first exposure to DataFrames was when I learnt about Pandas. Today, it is difficult for me to run my data science workflow with out Pandas DataFrames. So, when I saw similar functionality in Apache Spark, I was excited about the possibilities it opens up!

https://www.analyticsvidhya.com/blog/2016/10/spark-dataframe-and-operations/



REST
Web Application

Full Stack Application

Full Stack Developer Meaning & Definition

In technology development, full stack refers to an entire computer system or application from the front end to the back end and the code that connects the two. The back end of a computer system encompasses "behind-the-scenes" technologies such as the database and operating system. The front end is the user interface (UI). This end-to-end system requires many ancillary technologies such as the network, hardware, load balancers, and firewalls.

FULL STACK WEB DEVELOPERS

Full stack is most commonly used when referring to web developers. A full stack web developer works with both the front and back end of a website or application. They are proficient in both front-end and back-end languages and frameworks, as well as server, network, and hosting environments.

Full-stack developers need to be proficient in languages used for front-end development such as HTML, CSS, JavaScript, and third-party libraries and extensions for Web development such as JQuery, SASS, and REACT. Mastery of these front-end programming languages will need to be combined with knowledge of UI design as well as customer experience design for creating optimal front-facing websites and applications.

https://www.webopedia.com/definitions/full-stack/

Full Stack Web Developer

A full stack web developer is a person who can develop both **client** and **server** software.

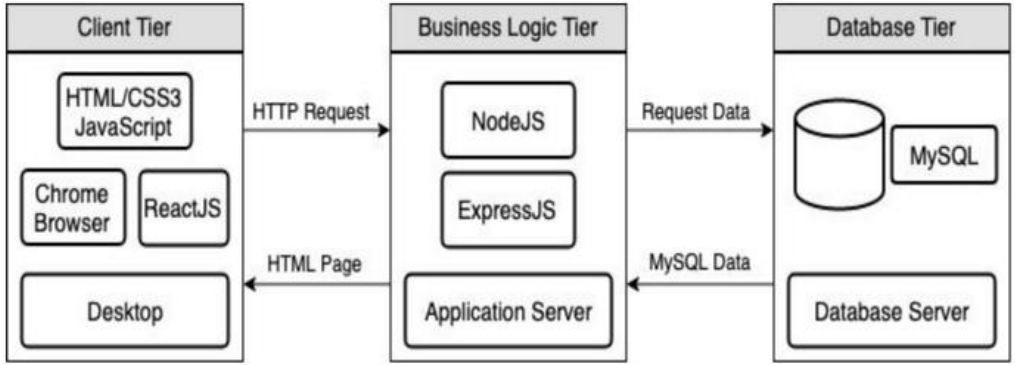
In addition to mastering HTML and CSS, he/she also knows how to:

- Program a **browser** (like using JavaScript, jQuery, Angular, or Vue)
- Program a **server** (like using PHP, ASP, Python, or Node)
- Program a **database** (like using SQL, SQLite, or MongoDB)

https://www.w3schools.com/whatis/whatis_fullstack.asp

- There are courses that cover topics:
  - COMS W4153: Advanced Software Engineering
  - COMS W4111: Introduction to Databases
  - COMS W4170 - User Interface Design
- This course will focus on cloud realization, microservices and application patterns, … …
- Also, I am not great at UIs … … We will not emphasize or require a lot of UI work.

Full Stack Web Application



M = Mongo
E = Express
R = React
N = Node

I start with FastAPI and MySQL, but all the concepts are the same.

https://levelup.gitconnected.com/a-complete-guide-build-a-scalable-3-tier-architecture-with-mern-stack-es6-ca129d7df805

- My preferences are to replace React with Angular, and Node with Flask.
- There are three projects to design, develop, test, deploy, … …
  1. Browser UI application.
  2. Microservice.
  3. Database.
- We will initial have two deployments: local machine, virtual machine. We will ignore the database for step 1.

Some Terms

- A web application server or web application framework: "A web framework (WF) or web application framework (WAF) is a software framework that is designed to support the development of web applications including web services, web resources, and web APIs. Web frameworks provide a standard way to build and deploy web applications on the World Wide Web. Web frameworks aim to automate the overhead associated with common activities performed in web development. For example, many web frameworks provide libraries for database access, templating frameworks, and session management, and they often promote code reuse." (https://en.wikipedia.org/wiki/Web_framework)

- REST: "REST (Representational State Transfer) is a software architectural style that was created to guide the design and development of the architecture for the World Wide Web. REST defines a set of constraints for how the architecture of a distributed, Internet-scale hypermedia system, such as the Web, should behave. The REST architectural style emphasises uniform interfaces, independent deployment of components, the scalability of interactions between them, and creating a layered architecture to promote caching to reduce user-perceived latency, enforce security, and encapsulate legacy systems.[1]

  REST has been employed throughout the software industry to create stateless, reliable web-based applications." (https://en.wikipedia.org/wiki/REST)

Some Terms

- OpenAPI: “The OpenAPI Specification, previously known as the Swagger Specification, is a specification for a machine-readable interface definition language for describing, producing, consuming and visualizing web services.” (https://en.wikipedia.org/wiki/OpenAPI_Specification)

- Model: “A model represents an entity of our application domain with an associated type.” (https://medium.com/@nicola88/your-first-openapi-document-part-ii-data-model-52ee1d6503e0)

- Routers: “What fastapi docs says about routers: If you are building an application or a web API, it’s rarely the case that you can put everything on a single file. FastAPI provides a convenience tool to structure your application while keeping all the flexibility.” (https://medium.com/@rushikeshnaik779/routers-in-fastapi-tutorial-2-adf3e505fdca)

- Summary:
  - These are general concepts, and we will go into more detail in the semester.
  - FastAPI is a specific technology for Python.
  - There are many other frameworks applicable to Python, NodeJS/TypeScript, Go, C#, Java, … …
  - They all surface similar concepts with slightly different names.

Data Modeling Concepts and REST

Almost any data model has the same core concepts:

- Types and instances:
  - Entity Type: A definition of a type of thing with properties and relationships.
  - Entity Instance: A specific instantiation of the Entity Type
  - Entity Set Instance: An Entity Type that:
    - Has properties and relationships like any entity, but …
    - Has at least one *special relationship* – ***contains.***
- Operations, minimally CRUD, that manipulate entity types and instances:
  - Create
  - Retrieve
  - Update
  - Delete
  - Reference/Identify/… …
  - Host/database/table/pk

REST (https://restfulapi.net/)

What is REST

- REST is acronym for **RE**presentational **S**tate **T**ransfer. It is architectural style for **distributed hypermedia systems** and was first presented by Roy Fielding in 2000 in his famous dissertation.
- Like any other architectural style, REST also does have it's own 6 guiding constraints which must be satisfied if an interface needs to be referred as **RESTful**. These principles are listed below.

Guiding Principles of REST

- **Client–server** – By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.
- **Stateless** – Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cacheable** – Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- **Uniform interface** – By applying the software engineering principle of generality to the component interface, the overall system architecture is simplified and the visibility of interactions is improved. In order to obtain a uniform interface, multiple architectural constraints are needed to guide the behavior of components. REST is defined by four interface constraints: identification of resources; manipulation of resources through representations; self-descriptive messages; and, hypermedia as the engine of application state.
- **Layered system** – The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot "see" beyond the immediate layer with which they are interacting.
- **Code on demand (optional)** – REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented.

Resources

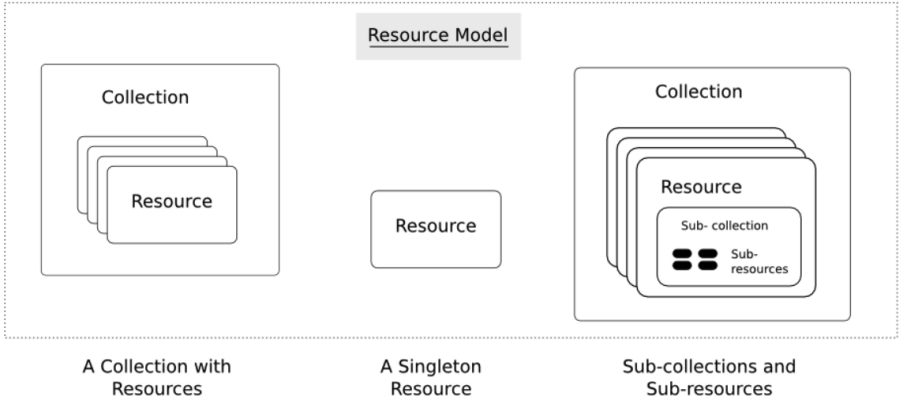
Resources are an abstraction. The application maps to create things and actions.

"A resource-oriented API is generally modeled as a resource hierarchy, where each node is either a *simple resource* or a *collection resource*. For convenience, they are often called a resource and a collection, respectively.

- A collection contains a list of resources of **the same type**. For example, a user has a collection of contacts.
- A resource has some state and zero or more sub-resources. Each sub-resource can be either a simple resource or a collection resource.

For example, Gmail API has a collection of users, each user has a collection of messages, a collection of threads, a collection of labels, a profile resource, and several setting resources.

While there is some conceptual alignment between storage systems and REST APIs, a service with a resource-oriented API is not necessarily a database, and has enormous flexibility in how it interprets resources and methods. For example, creating a calendar event (resource) may create additional events for attendees, send email invitations to attendees, reserve conference rooms, and update video conference schedules. (Emphasis added) (https://cloud.google.com/apis/design/resources#resources)



https://restful-api-design.readthedocs.io/en/latest/resources.html

REST – Resource Oriented

- When writing applications, we are used to writing functions or methods:
  - openAccount(last_name, first_name, tax_payer_id)
  - account.deposit(deposit_amount)
  - account.close()

  We can create and implement whatever functions we need.
- REST only allows four methods:
  - POST: Create a resource
  - GET: Retrieve a resource
  - PUT: Update a resource
  - DELETE: Delete a resource

  That's it. That's all you get.

"The key characteristic of a resource-oriented API is that it emphasizes resources (data model) over the methods performed on the resources (functionality). A typical resource-oriented API exposes a large number of resources with a small number of methods." (https://cloud.google.com/apis/design/resources)

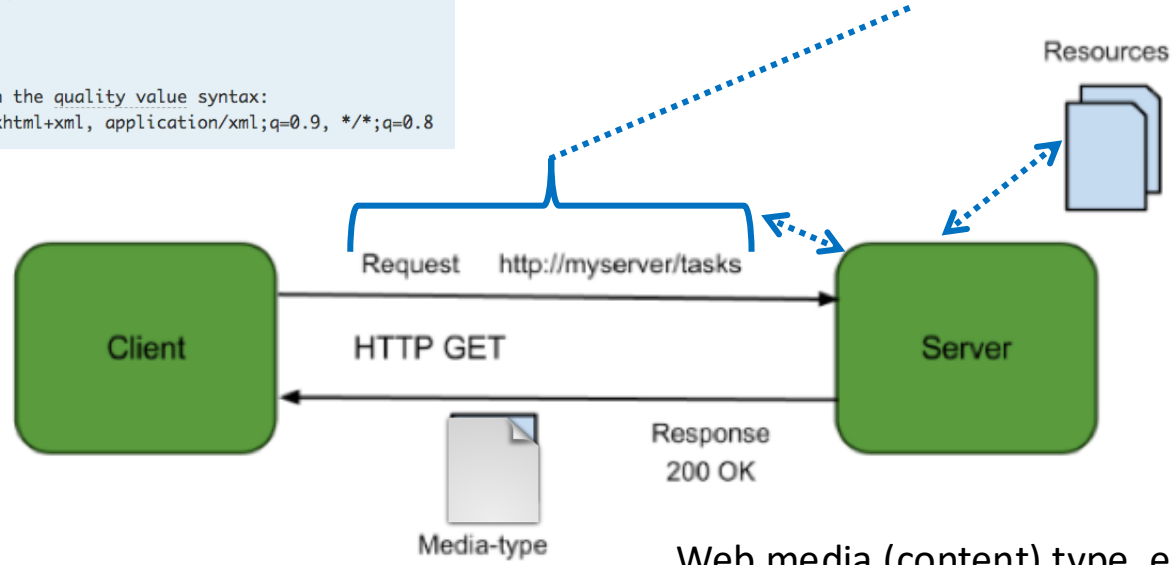
- A REST client needs no prior knowledge about how to interact with any particular application or server beyond a generic understanding of hypermedia.

Resources, URLs, Content Types

Accept type in headers.

```
Accept: <MIME_type>/<MIME_subtype>
Accept: <MIME_type>/*
Accept: */*

// Multiple types, weighted with the quality value syntax:
Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8
```

- Relative URL identifies “resource” on the server.
- Server implementation maps abstract resource to tangible “thing,” file, DB row, … and any application logic.



Client may be

Browser

Mobile device

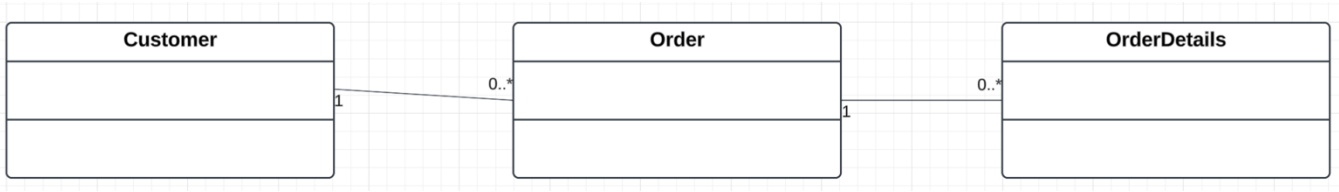
Other REST Service

… …

Web media (content) type, e.g.

- text/html
- application/json

Resources and APIs

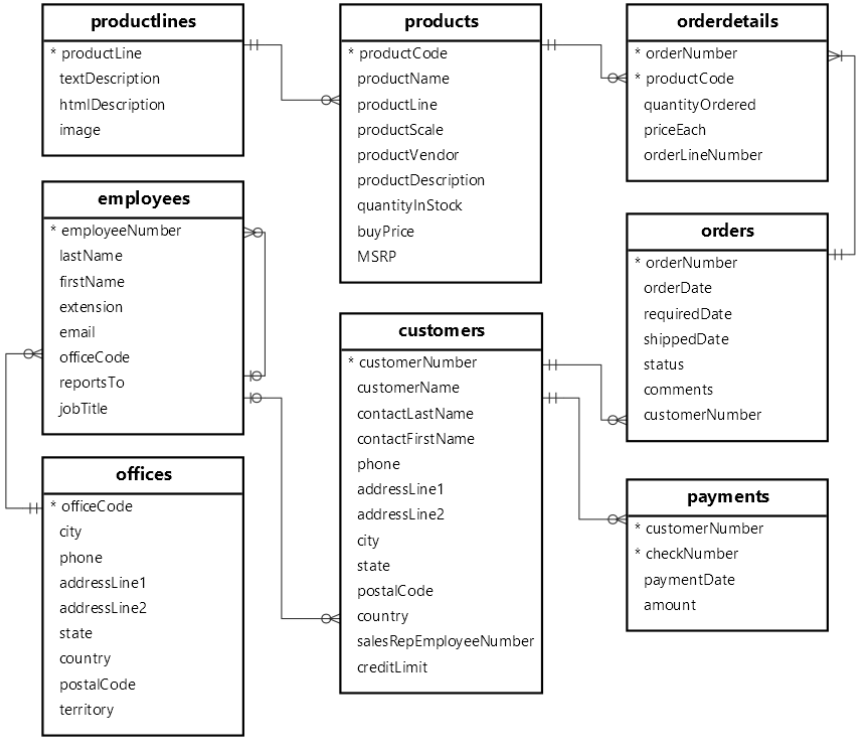


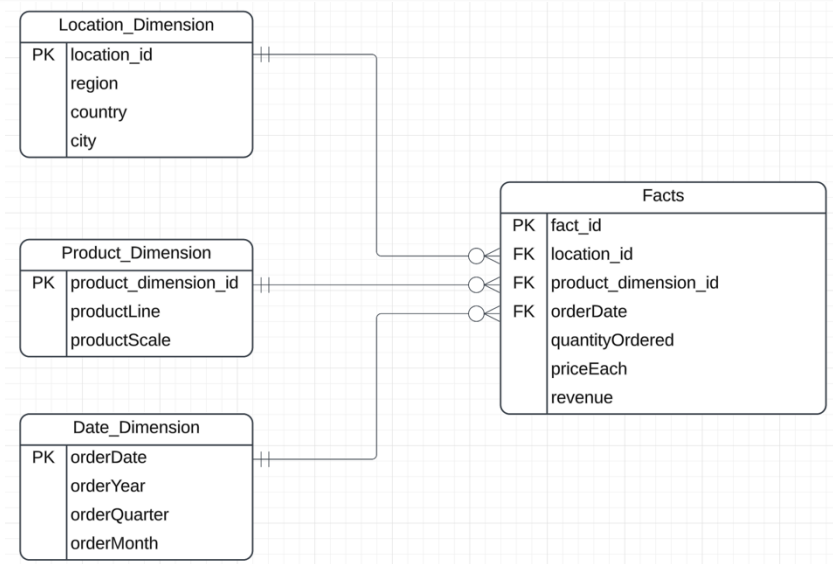
- You will implement 3 resources and map to Classicmodels: *Customers, Orders, OrderDetails*
- You will implement the following paths and methods
  - /customers: GET, POST
  - /customers/{customerNumber}: GET, PUT, DELETE
  - /customers/{customerNumber}/orders: GET, POST
  - /orders: GET, POST
  - /orders/{orderNumber}: GET, PUT, DELETE
  - /orders/{orderNumber}/orderdetails: GET, POST
  - /orders/{orderNumber}/orderdetails/{orderlinenumber}: GET, PUT, DELETE
- GET on the “collections” /customers, /orders, /orders{ordernumber/orderdetails will support query parameters, e.g.
  - GET /customers?country=France&city=Paris
  - GET /orders/12345/orderdetails?productCode=xyz1023
  - etc.
  - You will “translate” the query parameters into the proper SQL SELECT * FROM table WHERE … …

Homework 3

Non-Programming
Data Engineering, ROLAP

Classicmodels ➔ Star Schema





- Define a star schema.
- Load the schema from classicmodels data.
- Perform some basic queries and operations.

Define the Facts and Dimensions

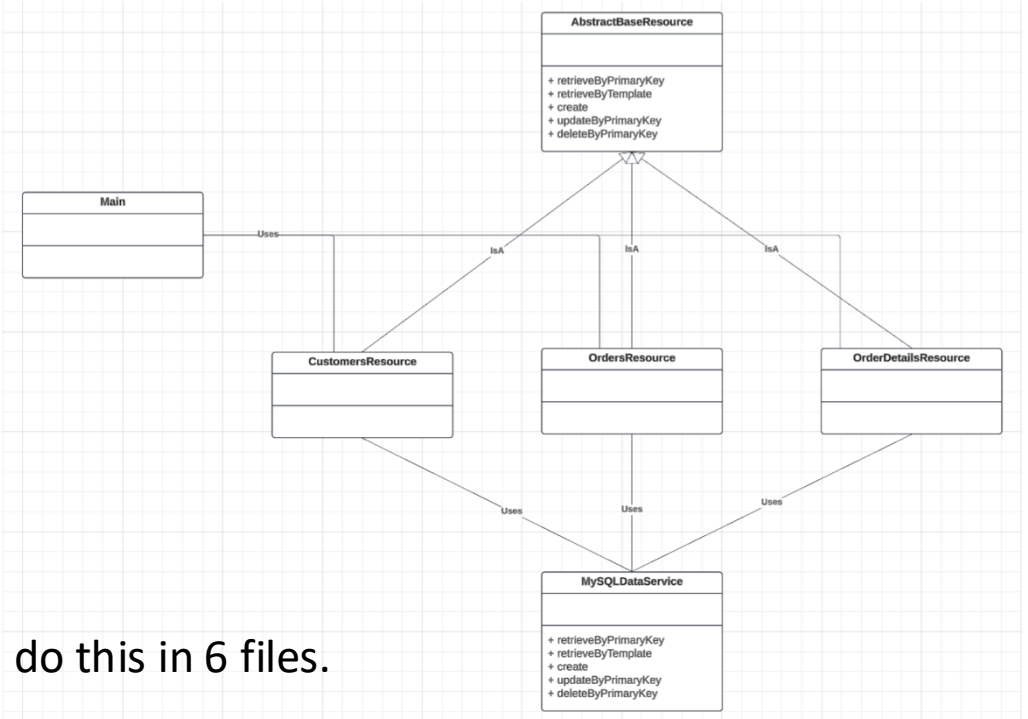
- The facts are:
  - (quantityOrdered, priceEach, revenue=quantityOrdered*priceEach)
- The dimensions are:
  - Location(region, country, city) (location of customer placing the order)
  - Date(year, quarter, month)
  - Product(productLine, productScale)
- The necessary information is in the tables:
  - Customers
  - Orders
  - Orderdetails
  - Products

Switch to Notebook for Non-Programming HW4

Programming

File and Class/Resource Model

- Main
  - Implements the routes.
  - Delegates onto the resource implementation class.
- Resource implementation class
  - Is trivial.
  - Simply delegates onto the data service.
- MySQLDataService
  - For each method
  - “Translates” the method signature into SQL
  - Calls MySQL
  - Returns the result



I do this in 6 files.

Implementation

- Use FastAPI (https://fastapi.tiangolo.com/)

- You can load the data from the slides with some additional writeup into Cursor, ChatGPT, etc. and it will generate the structure and implementation for you.

- You can write simple scripts in a Jupyter notebook to test your application and APIs using the requests (https://requests.readthedocs.io/en/latest/) framework.

- I am not hung up on perfections, good code, … …
  I just want to make sure that you can get something working.

Wrap Up